SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Abinesh H	**Reg. No.:**	192424387
Section / Batch:	B.Tech AIDS / 3rd Year	**Date:**	29/08/2026

Code of Conduct

I Abinesh H(192424387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Abinesh H_____

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:
"Artificial intelligence is"
the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:
"The cat sits on the mat."
and
"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	4	4	4	3	3	18	87
2	4	4	4	3	3	18	
3	4	3	3	4	3	17	
4	3	4	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 29/08/2026	Date: _____	Date: _____

Assignment: N-gram Language Modeling, Backoff, Entropy, and POS Tagging

---

1. N-gram Based Text Prediction

Introduction

An N-gram language model predicts the next word in a sentence by analyzing the previous words. A **unigram** considers one word, a **bigram** considers two consecutive words, and a **trigram** considers three consecutive words. The probability of an N-gram is calculated using its frequency in the corpus.

For example, for the incomplete sentence:

"Artificial intelligence is"

the model examines the corpus and predicts the most probable words that can follow **"is"**.

Methodology

The system performs the following operations:

1. Read an English text corpus.
2. Convert all text to lowercase.
3. Segment the corpus into sentences.
4. Tokenize sentences into individual words.
5. Generate unigram, bigram, and trigram frequency counts.
6. Calculate maximum-likelihood probabilities.
7. Predict the top-5 next words.
8. Test the model with multiple incomplete sentences.
9. Calculate prediction accuracy.
10. Demonstrate the zero-probability problem for unseen N-grams.

Maximum-Likelihood Probability

For a bigram:

$$P(w_i \mid w_{i-1}) = \frac{Count(w_{i-1}, w_i)}{Count(w_{i-1})}$$

For a trigram:

$$P(w_i, \mid w_{i-2}w_{i-1}) = \frac{Count(w_{i-2}, w_{i-1}, w_i)}{Count(w_{i-2}, w_{i-1})}$$

Python Implementation

```
import re
from collections import Counter
corpus = """
Artificial intelligence is changing the world.
Artificial intelligence is useful in many applications.
Machine learning is a part of artificial intelligence.
Machine learning can solve complex problems.
Natural language processing helps computers understand language.
Deep learning is used in artificial intelligence.
The computer can learn from data.
The model predicts the next word.
Language models predict words from context.
Artificial intelligence and machine learning are powerful technologies.
"""
# Preprocessing
sentences = re.split(r'[.!?]+', corpus.lower())
sentences = [re.findall(r'\b\w+\b', s) for s in sentences if s.strip()]
# N-gram counts
unigram = Counter()
bigram = Counter()
trigram = Counter()
for s in sentences:
    unigram.update(s)
    bigram.update(zip(s, s[1:]))
    trigram.update(zip(s, s[1:], s[2:]))
# Prediction
def predict(text, n):
    words = re.findall(r'\b\w+\b', text.lower())
    if n == 1:
        total = sum(unigram.values())
        result = [(w, c/total) for w, c in unigram.items()]
    elif n == 2:
        last = words[-1]
```

```
        total = unigram[last]
        result = [(w, c/total)
                for (a, w), c in bigram.items()
                if a == last and total > 0]
    else:
        context = tuple(words[-2:])
        total = bigram[context]
        result = [(w, c/total)
                for (a, b, w), c in trigram.items()
                if (a, b) == context and total > 0]
    return sorted(result, key=lambda x: x[1], reverse=True)[:5]
print("Unigram:", predict("Artificial intelligence is", 1))
print("Bigram :", predict("Artificial intelligence is", 2))
print("Trigram:", predict("Artificial intelligence is", 3))
```

Unseen N-gram

If an N-gram does not occur in the training corpus, its frequency is zero. Therefore:

$$P(N\text{-gram}) = 0$$

For example, if **"quantum processor"** does not occur in the corpus, the unsmoothed bigram model assigns it zero probability.

Limitations

The unsmoothed N-gram model has the following limitations:

- Unseen N-grams receive zero probability.
- It requires a large training corpus.
- It cannot understand the meaning of words.
- It has limited ability to capture long-distance relationships.
- Prediction quality decreases when the test sentence contains unfamiliar combinations.

Conclusion

The N-gram model provides a simple statistical method for next-word prediction. Unigrams provide general word frequencies, while bigrams and trigrams provide contextual information. However, the unsmoothed model suffers from the **zero-probability problem**, which motivates the use of backoff and interpolation techniques.

Output:

```
Unigram: [('artificial', 0.07462686567164178), ('intelligence', 0.07462686567164178),
('is', 0.05970149253731343), ('the', 0.05970149253731343), ('learning', 0.05970149253731343)]
Bigram : [('changing', 0.25), ('useful', 0.25), ('a', 0.25), ('used', 0.25)]
Trigram: [('changing', 0.5), ('useful', 0.5)]
```

2. Backoff and Interpolation for Language Prediction

Introduction

An unsmoothed N-gram model cannot predict an unseen N-gram because it assigns it zero probability.

Backoff solves this problem by using a lower-order model when a higher-order model is unavailable.

For example:

Machine learning can ...

The system first checks the trigram. If it is unavailable, it checks the bigram and finally the unigram.

Deleted Interpolation combines unigram, bigram, and trigram probabilities instead of selecting only one model.

Backoff Strategy

Check Trigram

↓

Available?

Yes → Use Trigram

No

↓

Check Bigram

↓

Available?

Yes → Use Bigram

No

↓

Use Unigram

Python Implementation

```
import re
from collections import Counter
corpus = """
Machine learning can solve problems.
Machine learning can analyze data.
Artificial intelligence can learn patterns.
Deep learning can process images.
Machine learning is useful.
Artificial intelligence is useful.
Natural language processing can understand text.
"""
sentences = [
    re.findall(r'\b\w+\b', s.lower())
    for s in re.split(r'[.!?]+', corpus) if s.strip()
]
uni = Counter()
bi = Counter()
tri = Counter()
for s in sentences:
    uni.update(s)
    bi.update(zip(s, s[1:]))
    tri.update(zip(s, s[1:], s[2:]))
def unsmoothed(words):
    context = tuple(words[-2:])
    total = bi[context]
    result = [(w, c/total)
              for (a, b, w), c in tri.items()
              if (a, b) == context and total > 0]
    return sorted(result, key=lambda x: x[1], reverse=True)[:5]
```

```
def backoff(words):
    result = unsmoothed(words)
    if result:
        return result
    last = words[-1]
    result = [(w, c/uni[last])
              for (a, w), c in bi.items()
              if a == last and uni[last] > 0]
    if result:
        return sorted(result, key=lambda x: x[1],
                      reverse=True)[:5]
    total = sum(uni.values())
    return sorted(
        [(w, c/total) for w, c in uni.items()],
        key=lambda x: x[1], reverse=True
    )[:5]
def interpolation(words):
    last = words[-1]
    context = tuple(words[-2:])
    total = sum(uni.values())
    scores = {}
    for w in uni:
        p1 = uni[w] / total
        p2 = bi[(last, w)] / uni[last] if uni[last] else 0
        p3 = tri[(context[0], context[1], w)] / bi[context] \
             if bi[context] else 0
        scores[w] = 0.2*p1 + 0.3*p2 + 0.5*p3
    return sorted(scores.items(),
                  key=lambda x: x[1],
                  reverse=True)[:5]
text = "machine learning can"
words = text.split()
```

print("Unsmoothed:", unsmoothed(words))

print("Backoff:", backoff(words))

print("Interpolation:", interpolation(words))

Comparison

Model	Zero Probability	Coverage	Prediction
Unsmoothed	High	Low	Limited
Backoff	Reduced	High	Better
Interpolation	Very Low	Very High	More robust

Conclusion

Backoff and interpolation improve N-gram language models by reducing the effect of sparse training data. Backoff selects a lower-order model when necessary, while interpolation combines information from multiple N-gram levels. Therefore, both methods provide better coverage and more reliable predictions.

Output:

```
Unsmoothed: [('solve', 0.3333333333333333), ('analyze', 0.3333333333333333), ('process', 0.3333333333333333)]
Backoff: [('solve', 0.3333333333333333), ('analyze', 0.3333333333333333), ('process', 0.3333333333333333)]
Interpolation: [('solve', 0.23254901960784313), ('analyze', 0.23254901960784313), ('process', 0.23254901960784313),
('learn', 0.06588235294117648), ('understand', 0.06588235294117648)]
```

---

3. Entropy-Based Evaluation of Language Models

Introduction

Entropy measures the uncertainty or unpredictability of a language model. A low entropy value indicates that the model can predict the words with greater confidence, whereas a high entropy value indicates greater uncertainty.

For example:

"The cat sits on the mat."

is likely to be more predictable than:

"The quantum processor redesigned the..."

if the training corpus mainly contains common sentences.

Methodology

The system:

1. Creates separate training and testing corpora.
2. Builds unigram, bigram, and trigram models.
3. Calculates word probabilities.
4. Calculates entropy for each test sentence.
5. Compares unsmoothed and smoothed models.
6. Classifies sentences as relatively high or low uncertainty.

Entropy Formula

$$H = -\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i \mid context)$$

Python Implementation

```
import math
import re
from collections import Counter
train = """
The cat sits on the mat.
The dog sits on the floor.
The cat eats food.
The dog eats food.
The cat runs fast.
"""
test = [
    "The cat sits on the mat",
    "The quantum processor redesigned the"
]
sentences = [
    re.findall(r'\b\w+\b', s.lower())
    for s in train.split(".") if s.strip()
]
uni = Counter()
bi = Counter()
```

```
tri = Counter()
for s in sentences:
    uni.update(s)
    bi.update(zip(s, s[1:]))
    tri.update(zip(s, s[1:], s[2:]))
V = len(uni)
def probability(word, context, n, smooth=False):
    if n == 1:
        if smooth:
            return (uni[word] + 1) / (sum(uni.values()) + V)
        return uni[word] / sum(uni.values())
    if n == 2:
        count = bi[(context[-1], word)]
        total = uni[context[-1]]
    else:
        count = tri[(context[-2], context[-1], word)]
        total = bi[(context[-2], context[-1])]
    if smooth:
        return (count + 1) / (total + V)
    return count / total if total else 0
def entropy(sentence, n, smooth=False):
    words = re.findall(r'\b\w+\b', sentence.lower())
    values = []
    for i, word in enumerate(words):
        context = words[max(0, i-n+1):i]
        p = probability(word, context, n, smooth)
        if p > 0:
            values.append(-math.log2(p))
        else:
            return float("inf")
    return sum(values) / len(values)
for sentence in test:
    print("\nSentence:", sentence)
```

```
for n in [1, 2, 3]:
    print("N =", n, "Entropy =", entropy(sentence, n))
print("Smoothed Trigram:",
      entropy(sentence, 3, True))
```

Interpretation

- **Low entropy** means high predictability.
- **High entropy** means low predictability.
- Unigram models generally provide less contextual information.
- Bigram models consider the previous word.
- Trigram models consider two previous words.
- Smoothing prevents unseen N-grams from producing infinite/undefined entropy.

Conclusion

Entropy provides a quantitative measure of prediction uncertainty. A reliable language model should generally assign higher probabilities to likely words and therefore produce lower entropy on predictable text. Comparing entropy across N-gram models helps evaluate how effectively each model captures language patterns.

Output:

```
Sentence: I book a ticket
Rule Based: [('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]
Stochastic: [('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]
Transformation: [('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN')]

Sentence: I read a book
Rule Based: [('I', 'PRON'), ('read', 'NOUN'), ('a', 'DET'), ('book', 'NOUN')]
Stochastic: [('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]
Transformation: [('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]
```

---

4. Comparative POS Tagging System

Introduction

Part-of-Speech (POS) tagging assigns grammatical categories such as noun, verb, adjective, adverb, pronoun, and determiner to words.

The same word can have different POS tags depending on its context. For example:

I book a ticket. → *book = Verb*

I read a book. → *book = Noun*

Three approaches can be used:

1. Rule-based POS tagging
2. Stochastic POS tagging
3. Transformation-based POS tagging

4.1 Rule-Based POS Tagging

Rule-based tagging uses dictionaries and manually defined grammatical rules.

Example rules:

If word = I, he, she, they → PRON

If word = a, an, the → DET

If word ends with "ly" → ADV

If word follows a determiner → NOUN

If word follows a pronoun and is an action word → VERB

4.2 Stochastic POS Tagging

Stochastic tagging uses probabilities learned from a tagged corpus.

It considers:

- Word/tag frequency
- Tag frequency
- Tag transition probability
- Contextual probability

For example:

$$P(tag \mid word) = \frac{Count(w, ordtag)}{Count(word)}$$

4.3 Transformation-Based Tagging

Transformation-based tagging first assigns an initial tag and then applies correction rules.

Example:

Initial:

book → NOUN

Transformation:

If "book" follows a pronoun → change NOUN to VERB

Python Implementation

```
from collections import Counter
training = [
    ("I", "PRON"), ("book", "NOUN"),
    ("a", "DET"), ("ticket", "NOUN"),
    ("I", "PRON"), ("read", "VERB"),
    ("a", "DET"), ("book", "NOUN"),
    ("She", "PRON"), ("reads", "VERB"),
    ("the", "DET"), ("book", "NOUN"),
    ("They", "PRON"), ("book", "VERB")
]
word_tag = {}
for word, tag in training:
    word_tag.setdefault(word.lower(), Counter())[tag] += 1
def rule_based(words):
    result = []
    for w in words:
        x = w.lower()
        if x in ["i", "he", "she", "they", "we"]:
            tag = "PRON"
        elif x in ["a", "an", "the"]:
            tag = "DET"
        elif x in ["book", "ticket", "read"]:
            tag = "NOUN"
        else:
            tag = "NOUN"
        result.append((w, tag))
    return result
def stochastic(words):
    result = []
    for w in words:
        if w.lower() in word_tag:
            tag = word_tag[w.lower()].most_common(1)[0][0]
```

```
        else:
            tag = "NOUN"
        result.append((w, tag))
    return result
def transformation(words):
    result = stochastic(words)
    for i, (word, tag) in enumerate(result):
        if word.lower() == "book" and i > 0:
            if result[i-1][1] == "PRON":
                result[i] = (word, "VERB")
            elif result[i-1][1] == "DET":
                result[i] = (word, "NOUN")
    return result
sentences = [
    "I book a ticket",
    "I read a book"
]
for sentence in sentences:
    words = sentence.split()
    print("\nSentence:", sentence)
    print("Rule Based:", rule_based(words))
    print("Stochastic:", stochastic(words))
    print("Transformation:", transformation(words))
```

Comparison

Approach	**Principle**	**Advantages**	**Limitations**
Rule-Based	Hand-written rules	Simple and interpretable	Difficult to handle complex language
Stochastic	Statistical probabilities	Learns from data	Requires tagged corpus
Transformation-Based	Initial tags + correction rules	Context-sensitive	Rules must be carefully designed

Evaluation

The accuracy of a POS tagger can be calculated as:

$$Accuracy = \frac{Correctly\ Tagged\ Words}{Total\ Words} \times 100$$

For a larger implementation, a tagged corpus such as the **Brown Corpus** can be divided into training and testing sets to obtain a meaningful accuracy score.

Conclusion

The three POS tagging approaches differ in how they use linguistic information. Rule-based systems are easy to understand but require extensive manual rules. Stochastic systems learn patterns from training data and can handle statistical variations. Transformation-based systems combine initial tagging with contextual correction rules. Together, these approaches demonstrate how context can determine the grammatical role of a word.

Output:

```
Sentence: I book a ticket
Rule Based: [('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]
Stochastic: [('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]
Transformation: [('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN')]

Sentence: I read a book
Rule Based: [('I', 'PRON'), ('read', 'NOUN'), ('a', 'DET'), ('book', 'NOUN')]
Stochastic: [('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]
Transformation: [('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]
```